PERSONAL AZURE INFRASTRUCTURE

Infrastructure as Code with Terraform | Version 2

Hub-and-spoke network, modules, least-privilege NSGs, private storage, remote state, and cost-controlled optional services

Project	Personal Azure Infrastructure: IaC via Terraform
Version / date	Version 2, October 26, 2026 (supersedes Version 1 of September 27, 2026, which stays in the repo under docs/)
Author	Bryan Sykes
Platform	Microsoft Azure
IaC tool	Terraform 1.16.4 (Windows) / 1.16.3 (Ubuntu), provider hashicorp/azurerm v3.117.1
Execution environment	VS Code on a Windows desktop with Azure CLI v2.90.0
Remote state	Azure Blob Storage: TerraformStateRG / sykestfstate / tfstate
GitHub	github.com/MrBSykes/personal-azure-infrastructure-terraform-IAC

What changed since Version 1

Version 1 documented the core build. Version 2 documents the work done after it, and, just as important, explains each component in terms of what it does in Azure, which business problem it solves, and what I would do differently the next time I stood this environment up from scratch.

Area	Version 1 (Sept 27)	Version 2 (Oct 26)
Resources in state	17	20 (prod NSG, its subnet association, and prod storage added)
Code structure	One flat main.tf	Three modules (spoke-network, baseline-nsg, secure-storage); dev and prod defined as data in spokes.auto.tfvars; 14 resources moved with moved blocks and a zero-destroy plan
Prod spoke	VNet only	Baseline NSG and a locked-down storage account, added by editing data, not code
Dev storage	Documented as "public access denied"	A review found public network access still enabled for selected networks; fixed in code and re-verified in the portal (before and after screenshots)
Linux VM / Bastion	VM commented out; Bastion planned	Both written, behind enable_vm and enable_bastion switches that default to off; a plan with both on shows 5 resources to add; nothing deployed, so nothing is billed
Subscription	Free trial	Trial credit expired and the subscription was disabled; upgraded to pay-as-you-go; terraform plan then reported "No changes", proving state and resources survived

Corrections to Version 1

Reviewing Version 1 against the deployed environment turned up four statements that were wrong or overstated. They are corrected here rather than left to stand:

- **Hub inspection.** V1 said all spoke traffic routes through the hub for centralized inspection. Peering is non-transitive and no firewall or route table exists, so spokes simply cannot reach each other. That is

isolation, not inspection. Inspection would need Azure Firewall (or an appliance) plus user-defined routes.

- **GatewaySubnet and Bastion.** V1 described GatewaySubnet as reserved for VPN or Bastion. It is reserved for VPN and ExpressRoute gateways only. Bastion needs its own subnet named AzureBastionSubnet.
- **Private DNS.** V1 said the storage name resolves to the private IP inside the VNet. The private endpoint exists, but no private DNS zone is linked, so that resolution was never demonstrated. It is listed as a next step.
- **Subnet name.** V1 listed the prod subnet as "prod-subnet"; the deployed name is private-subnet, matching dev.

1. The Business Problem

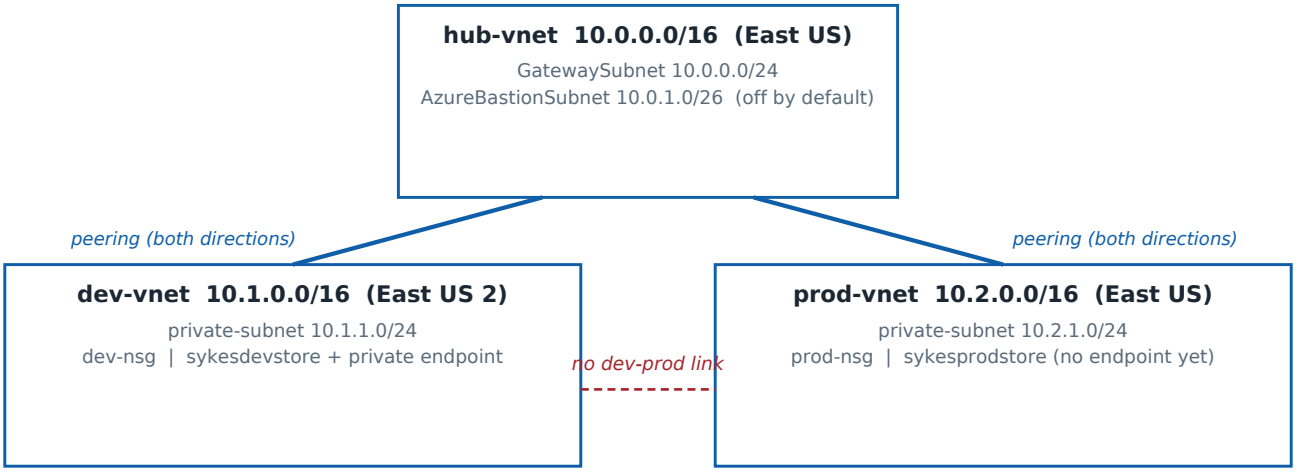
In federal IT and enterprise cloud work, infrastructure is rarely built once. The same environment types (development, staging, production, disaster recovery) get rebuilt across projects, contracts, and security boundaries. Built by hand in a portal, that creates three problems:

- **Inconsistency.** Two engineers clicking through the portal produce subtly different environments: different NSG rules, naming, TLS settings. Under FedRAMP or CMMC assessment, inconsistency is a finding.
- **No audit trail.** Portal changes leave no version-controlled record of what was built, when, or by whom.
- **Slow, error-prone rebuilds.** Recreating a network after an incident or a contract change takes days by hand. In Terraform it takes minutes and comes out identical.

Scenario	Without IaC	With Terraform
New contract award	Days of portal work	terraform apply, minutes
Security incident, rebuild an environment	Days to recreate; misconfiguration risk	terraform destroy and apply, identical result
Audit: prove configuration	Screenshots and manual notes	Git history is the record
Add a prod copy of dev controls	Repeat every click, hope nothing differs	Edit one data file (this project did exactly this)
Onboard a new engineer	Tribal knowledge	Read the .tf files

2. Architecture

Hub-and-spoke is the standard enterprise Azure layout: a central hub VNet holds shared services (gateways, Bastion, DNS, firewalls) and spoke VNets hold workloads. Spokes connect to the hub and never to each other.



Component	Name	Address space	Purpose
Hub VNet (East US)	hub-vnet	10.0.0.0/16	Shared services
Hub gateway subnet	GatewaySubnet	10.0.0.0/24	Reserved for a future VPN or ExpressRoute gateway
Hub Bastion subnet	AzureBastionSubnet	10.0.1.0/26	Created only when enable_bastion is true
Dev VNet (East US 2)	dev-vnet	10.1.0.0/16	Development workloads
Dev private subnet	private-subnet	10.1.1.0/24	NSG-enforced; hosts the storage private endpoint
Prod VNet (East US)	prod-vnet	10.2.0.0/16	Production workloads
Prod private subnet	private-subnet	10.2.1.0/24	NSG-enforced

Security design principles

- **Default deny.** Each private subnet denies all inbound traffic except what is explicitly allowed.
- **Private by default.** Storage accounts have public network access disabled; the only path in is a private endpoint.
- **Least privilege.** SSH is allowed only from the hub address range, never the internet.
- **Environment isolation.** Dev and prod each peer only to the hub, so there is no network path between them.
- **Encrypted transport.** TLS 1.2 minimum on every storage account.

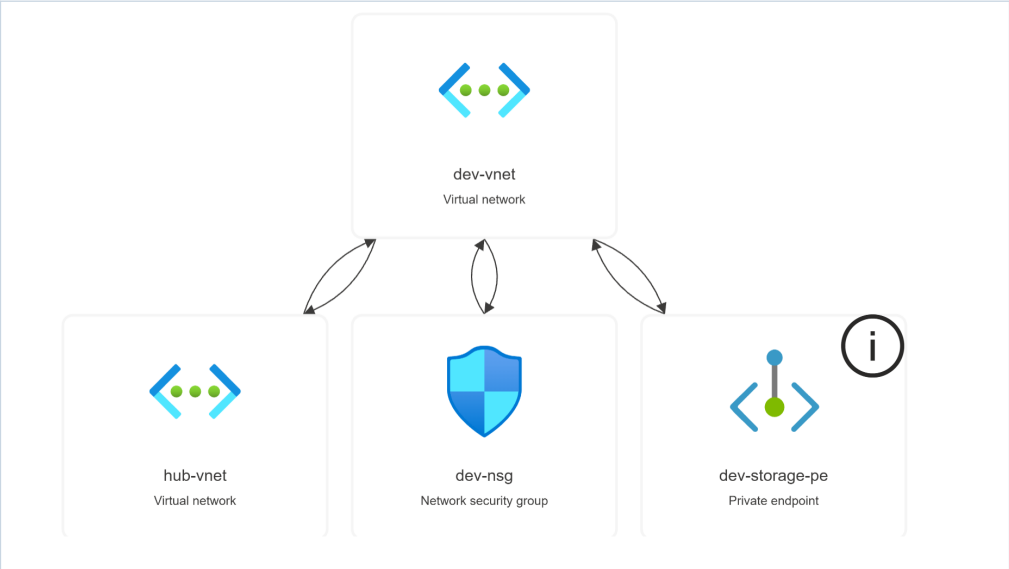


Figure 1. Azure resource visualizer: dev-vnet connected to hub-vnet, dev-nsg, and the dev-storage-pe private endpoint. (screenshots/05-dev-vnet-resource-visualizer.png)

3. Component Deep Dives

Each component below answers four questions: what it does in Azure, what business problem it solves, how it was built here, and what I would do differently on the next build.

3.1 Remote state

What it does in Azure	Terraform keeps a state file mapping its code to real resources. The azurerm backend stores that file in a blob container and takes a blob lease lock during changes, so two runs cannot write at the same time and a lost laptop does not lose the state.
Business problem	State is the record of what Terraform believes exists. On one machine it is a single point of failure and cannot be shared. Lose it and every resource becomes an orphan that code can no longer manage.
How it was built	The resource group, storage account, and container were created with the Azure CLI first, because Terraform cannot store state in a resource it has not yet created. The backend block points at TerraformStateRG / sykestfstate / tfstate with key dev.terraform.tfstate. Before the module refactor I took a backup with terraform state pull (74.8 KB) so there was a way back.
Next time	Use one state per environment so a bad dev apply cannot touch prod (today one file holds hub, dev, and prod). Turn on blob versioning and soft delete on the state account, and restrict access with RBAC instead of account keys.

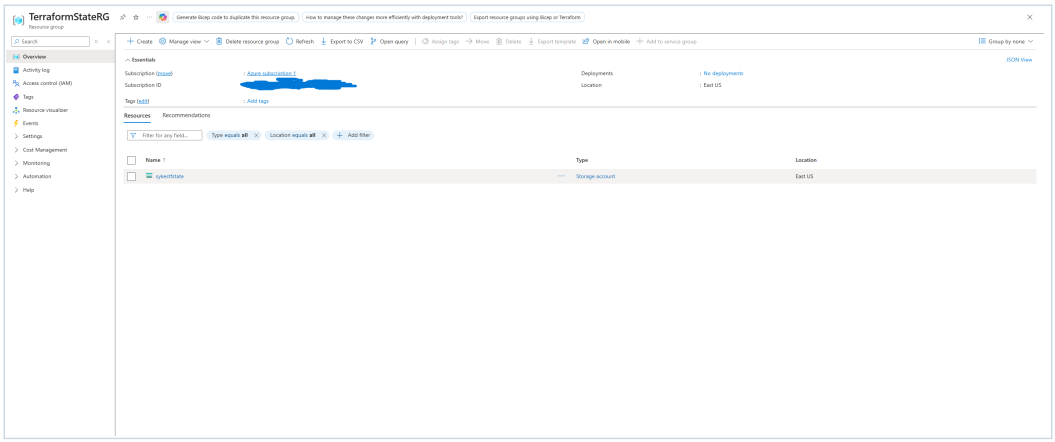


Figure 2. TerraformStateRG containing the sykestfstate storage account that holds Terraform state. (screenshots/04-terraform-state-rg.png)

3.2 Hub-and-spoke VNets and peering

What it does in Azure	A VNet is an isolated private network. Peering joins two VNets over the Microsoft backbone so they talk by private IP. It must be created from both sides, and it is non-transitive: if A and B both peer to C, A still cannot reach B.
Business problem	Environments must be isolated from each other yet share services (remote access, DNS, firewalls) without duplicating them per environment. Overlapping address ranges make later connections impossible, so the IP plan has to come first.
How it was built	Three non-overlapping /16 ranges (10.0, 10.1, 10.2). The spoke-network module creates the spoke VNet and both peering directions, so every spoke gets a correct, complete pair. Hub and dev are in different regions (East US and East US 2), which makes that link global peering.
Next time	Put everything in one region. The split exists only because of VM capacity trouble, and global peering bills data transfer per GB. Keep an address-plan table, and reserve the Bastion and firewall subnets in the hub from day one.

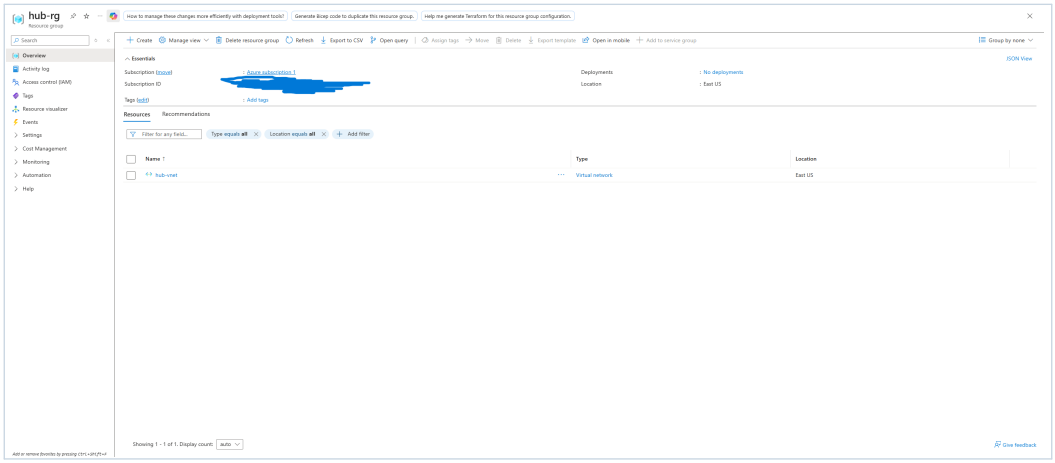


Figure 3. hub-rg with hub-vnet. (screenshots/02-hub-rg-resources.png)

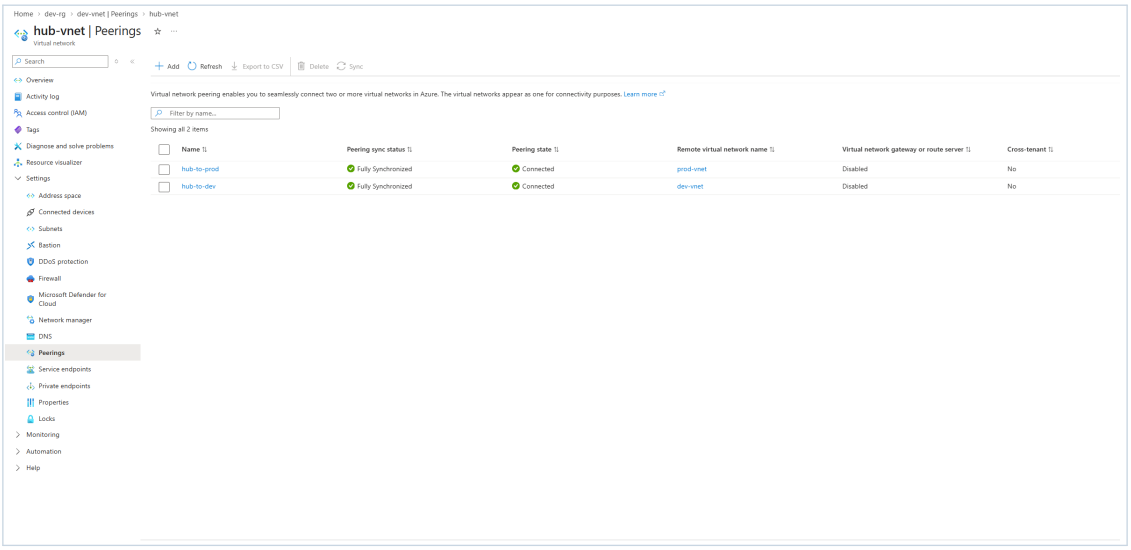


Figure 4. Hub peerings to prod and dev: Connected and Fully Synchronized. (screenshots/08-hub-vnet-peerings.png)

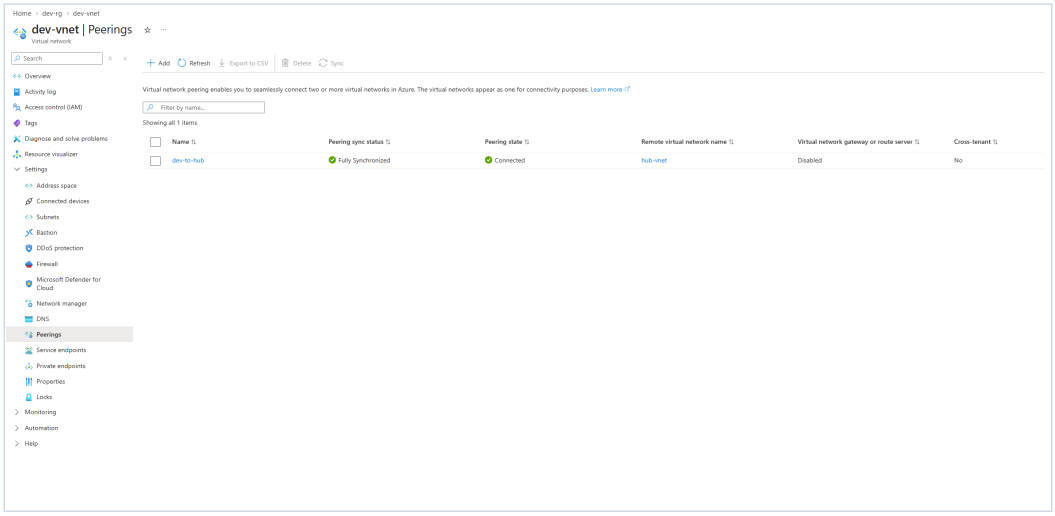


Figure 5. dev-to-hub peering: Connected and Fully Synchronized. (screenshots/09-dev-vnet-peerings.png)

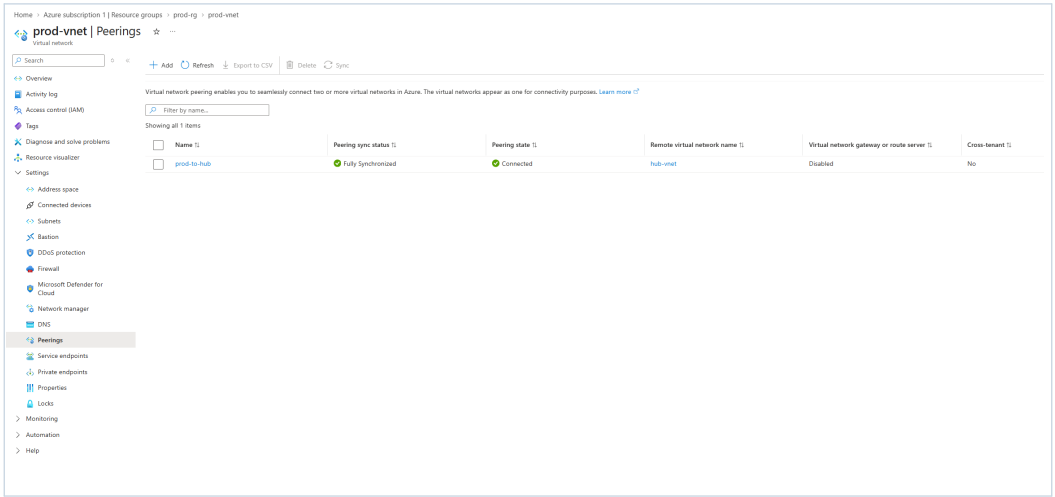


Figure 6. prod-to-hub peering: Connected and Fully Synchronized. Together with Figures 3 and 4, all four peering resources are visible and bidirectional. (screenshots/10-prod-vnet-peerings.png)

3.3 Baseline network security group

What it does in Azure	An NSG is a stateful packet filter applied to a subnet. Rules are evaluated by priority, lowest number first, and the first match wins. Azure also adds default rules at priorities 65000 and up.
Business problem	On a flat network, one compromised host can reach everything. Subnet-level least privilege limits the blast radius and gives auditors a short, readable rule list.
How it was built	Priority 100 allows SSH (22) only from 10.0.0.0/16, priority 200 allows HTTPS outbound, and priority 4096 denies all other inbound. The explicit deny matters: Azure treats peered networks as part of the VirtualNetwork tag, so the default AllowVnetInBound rule at 65000 would let the hub reach every port. The deny at 4096 is evaluated first and closes that. The rules live in the baseline-nsg module, so dev and prod are guaranteed identical.
Next time	Outbound is not actually default-deny: the default AllowInternetOutBound (65001) is still in force, so the rule at 200 documents intent but restricts nothing. Add an explicit deny-all outbound and allow only what is needed, turn on NSG flow logs, and use application security groups once VMs exist.

Rule	Priority	Direction	Source	Port	Action
allow-ssh-inbound	100	Inbound	10.0.0.0/16 (hub only)	22	Allow
allow-https-outbound	200	Outbound	Any (to Internet)	443	Allow
deny-all-inbound	4096	Inbound	Any	Any	Deny

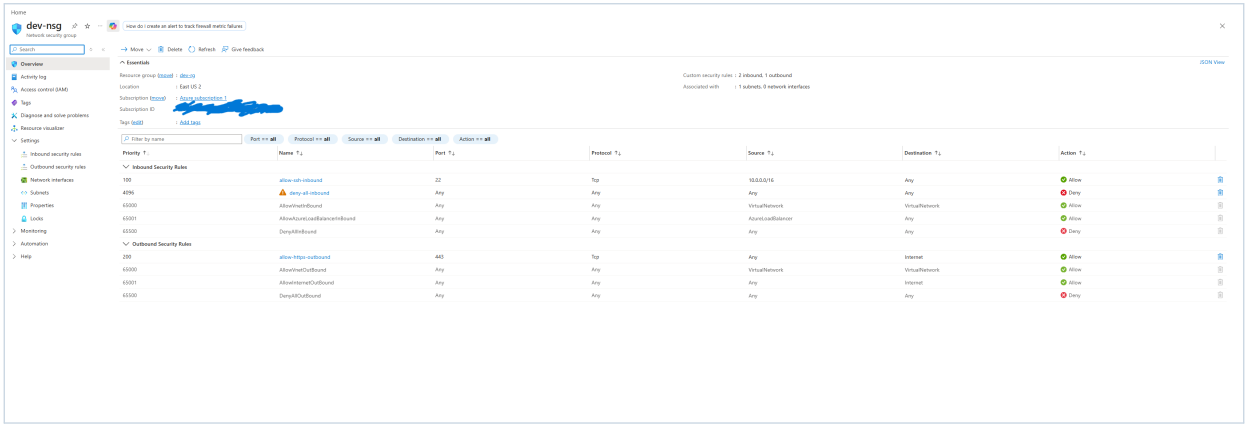


Figure 7. dev-nsg inbound and outbound rules, including the explicit deny-all-inbound at priority 4096. (screenshots/06-dev-nsg-security-rules.png)

3.4 Secure storage and the private endpoint

What it does in Azure	Storage accounts are reachable from the internet by default. Network rules can default-deny, and a private endpoint places a network interface with a private IP from your subnet in front of one service (blob) of the account. Setting public network access to Disabled leaves only private endpoints able to connect.
Business problem	Publicly reachable storage and anonymous blob access are among the most common causes of cloud data exposure. Closing both at the resource level removes the exposure instead of relying on every user to behave.
How it was built	The secure-storage module sets TLS 1.2, default action Deny, public network access off, and anonymous blob access off, with an optional blob private endpoint (billed hourly plus per GB processed, so optional per environment).
What the review found	The portal showed the dev account with public network access enabled (scoped to selected networks) and anonymous blob access enabled, which contradicted the documentation. I fixed it in code; Terraform reported a 1 to change, in-place update (no replacement) and the portal then showed both settings Disabled with one private endpoint connection (Figures 11 and 12).

Open item: DNS	A private endpoint only works by name when a private DNS zone (privatelink.blob.core.windows.net) is linked to the VNet. That zone, the VNet link, and a DNS zone group are not yet configured, so private name resolution is not yet demonstrated. A private DNS zone has its own small charge.
Next time	Create the endpoint and its DNS zone as one unit from the start, disable shared key access and use Entra ID RBAC, and enable blob soft delete and versioning.

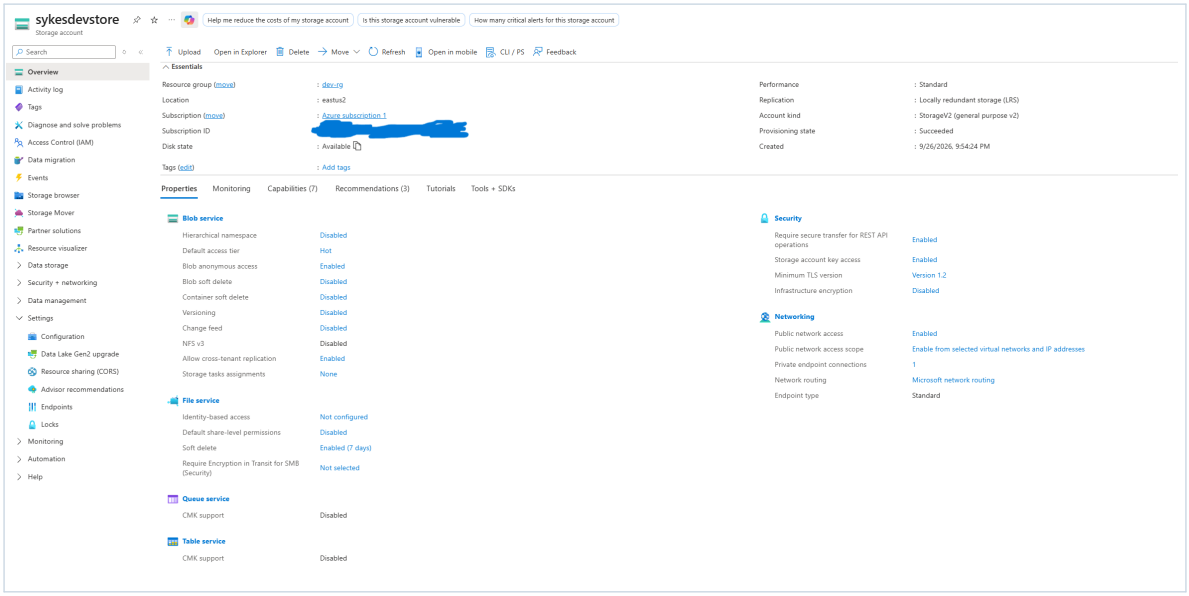


Figure 8. sykesdevstore BEFORE the fix: public network access and blob anonymous access both Enabled. (screenshots/07-sykesdevstore-BEFORE-lockdown.png)

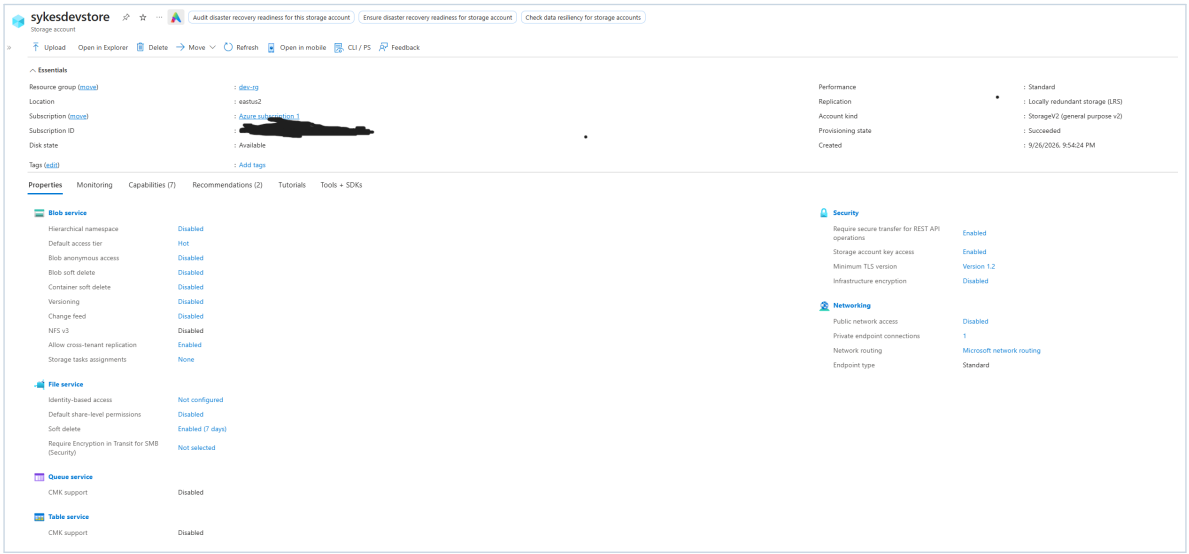


Figure 9. sykesdevstore AFTER the fix: public network access Disabled, blob anonymous access Disabled, one private endpoint connection. (screenshots/11-sykesdevstore-AFTER-lockdown.png)

3.5 Modules and spokes defined as data

What it does in Azure	A module is a reusable bundle of resources with inputs and outputs. for_each stamps it out once per environment. A moved block tells Terraform that a resource at an old code address is the same real object at a new address, so it renames the state entry instead of destroying and recreating the resource.

Business problem	Copy-pasted infrastructure drifts, and drift is exactly what configuration-management audits flag. A module changes once and applies everywhere, so "prod matches dev" is guaranteed by construction rather than discipline.
How it was built	Three modules, with dev and prod described in spokes.auto.tfvars. 14 of the 17 original resources moved; the 3 hub resources stayed put. Sequence: backup state, branch, validate, plan. The plan listed 14 resources as moved and ended **0 to add, 0 to change, 0 to destroy**. Only then was it applied, and a second plan reported "No changes".
Next time	Write the module before the second environment exists instead of retrofitting it. Version the modules, and run fmt, validate, and plan automatically on every pull request.

```
spokes = {
  dev  = { location = "eastus2", address_space = "10.1.0.0/16", subnet_prefix = "10.1.1.0/24",
           storage_account_name = "sykesdevstore", private_endpoint = true }
  prod = { location = "eastus",  address_space = "10.2.0.0/16", subnet_prefix = "10.2.1.0/24",
           storage_account_name = "sykesprodstore" }
}
```

3.6 Prod parity

What it does in Azure	The prod spoke receives the same baseline NSG and a locked-down storage account as dev.
Business problem	Production is where a misconfiguration costs the most. If prod uses the identical, already-reviewed controls, the security posture is tested before it ships.
How it was built	Because of the modules, this was a data-only change to spokes.auto.tfvars. The plan showed **3 to add, 1 to change, 0 to destroy**: the prod NSG, its association, the prod storage account, and the in-place dev storage fix. Prod has no private endpoint yet. It bills hourly, and I chose to document that trade-off instead of paying for it.
Next time	Build every environment from the module on day one so parity never has to be retrofitted.

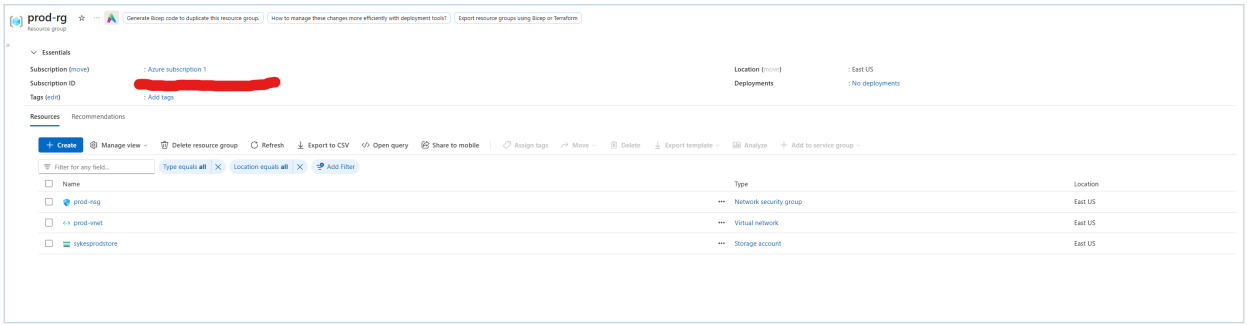


Figure 10. prod-rg after stage 2: prod-nsg, prod-vnet, and sykesprodstore. (screenshots/12-prod-rg-resources-UPDATED.png)

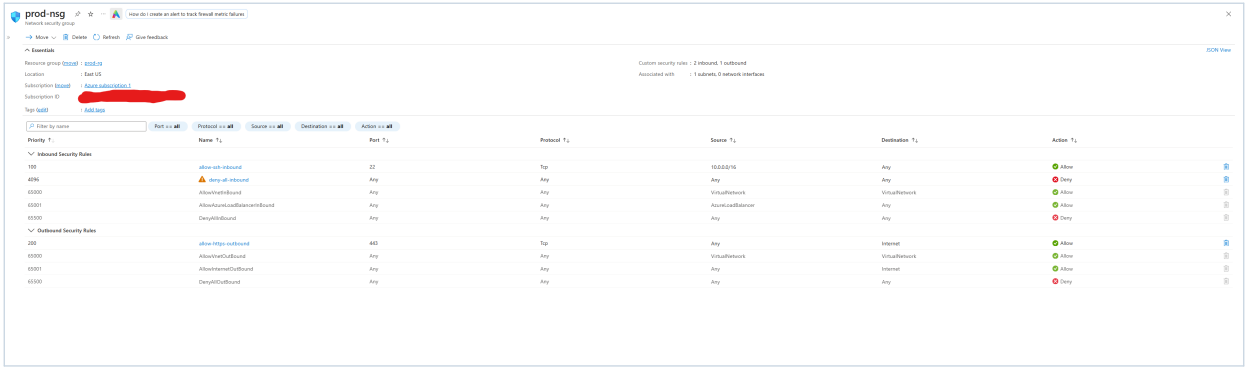


Figure 11. prod-nsg rules, identical to dev-nsg because both come from the same module. (screenshots/13-prod-nsg-security-rules.png)

3.7 Linux VM and Azure Bastion behind cost switches

What it does in Azure	Bastion is a managed service that gives browser-based SSH or RDP over TLS to VMs by private IP, so VMs need no public IP and no internet-facing management port. It lives in a dedicated subnet named AzureBastionSubnet (minimum /26).
Business problem	Remote administration is the front door attackers probe: exposed ports 22 and 3389. A managed access layer removes that door and gives one audited path in.
SKU decision	The Developer SKU is free but runs on a shared pool, allows a single connection, and does not support VNet peering, so it cannot reach VMs in a spoke from the hub. Reaching a peered spoke needs the Basic SKU or higher, which is dedicated and billed hourly from creation to deletion, plus a public IP.
How it was built	Two variables, enable_vm and enable_bastion, both default false, gate the resources with count. The VM has no public IP, uses key-only login (an RSA 4096 key, per issue 3), and a precondition rejects a plan that enables it without a key. A plan with both switches on shows **5 to add, 0 to change, 0 to destroy**: the Bastion subnet, its public IP, the Bastion host, the VM NIC, and the VM. It was not applied.
Why off by default	Anything that bills by the hour should be deliberate. The code is complete and reviewable, the plan is the evidence, and a timed demo (apply, capture, destroy in one sitting) is available when screenshots are needed.
Not yet validated	Whether a hub Bastion in East US reaches a VM in East US 2 across global peering can only be proven by a real apply, and the VM size needs to be confirmed deployable with az vm list-skus.
Next time	Single region, the Bastion subnet reserved in the address plan from day one, a budget alert before any paid resource, and an automatic teardown after demos.

4. Resources Deployed

20 Azure resources are deployed and tracked in Terraform state across 3 resource groups. The state storage account is separate, created with the CLI before Terraform.

Terraform address	Azure resource	Status
azurerm_resource_group.hub	hub-rg (East US)	Deployed
azurerm_virtual_network.hub	hub-vnet (10.0.0.0/16)	Deployed
azurerm_subnet.hub_gateway	GatewaySubnet (10.0.0.0/24)	Deployed
module.spoke_network["dev"] (5 resources)	dev-rg, dev-vnet, private-subnet, hub-to-dev and dev-to-hub peerings	Deployed
module.spoke_network["prod"] (5 resources)	prod-rg, prod-vnet, private-subnet, hub-to-prod and prod-to-hub peerings	Deployed
module.baseline_nsg["dev"] (2 resources)	dev-nsg and its association to the dev subnet	Deployed
module.baseline_nsg["prod"] (2 resources)	prod-nsg and its association to the prod subnet	Deployed
module.secure_storage["dev"] (2 resources)	sykesdevstore and dev-storage-pe private endpoint	Deployed
module.secure_storage["prod"] (1 resource)	sykesprodstore (no private endpoint)	Deployed
azurerm_network_interface / linux_virtual_machine (switch)	dev-vm-nic and dev-vm	Off (enable_vm)
azurerm_subnet / public_ip / bastion_host (switch)	AzureBastionSubnet, hub-bastion-pip, hub-bastion	Off (enable_bastion)

Count check: 3 hub + 5 dev network + 5 prod network + 2 dev NSG + 2 prod NSG + 2 dev storage + 1 prod storage = 20.

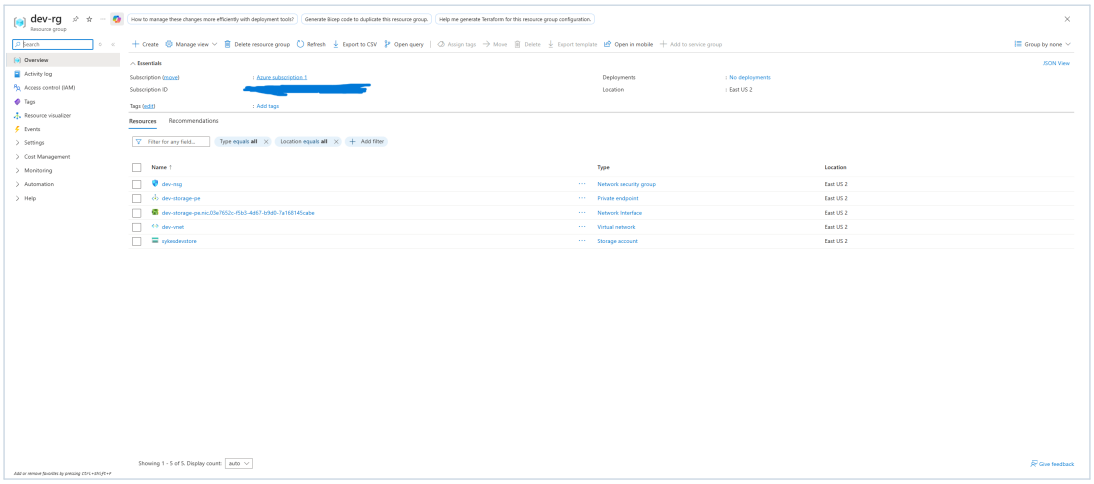


Figure 12. dev-rg: NSG, private endpoint and its NIC, VNet, and storage account. (screenshots/01-dev-rg-resources.png)

5. Troubleshooting Log

Every issue is recorded with its root cause, fix, and the lesson a team would carry forward. Issues 1 to 6 are carried over from Version 1; 7 to 10 are new.

#	Symptom	Root cause	Resolution	Lesson
1	terraform init: exec "az" not found	Azure CLI was on the Ubuntu box, not the Windows machine running Terraform	Installed Azure CLI v2.90.0 on Windows; restarted VS Code	Every machine that runs Terraform needs the full dependency stack; make it an onboarding prerequisite
2	Resource provider registration error on plan	Terraform tries to auto-register providers; the free account lacked permission	Registered Microsoft.Network, Compute, Storage with az provider register	Provider registration is a one-time subscription setup step for a platform team
3	ssh-ed25519 key not supported	Azure Linux VMs accept only RSA keys	Generated a dedicated RSA 4096 key for Azure	Keep separate key pairs per platform and track the key inventory
4	SkuNotAvailable for B1s and B1ms	No free-tier B-series capacity in East US or East US 2 at the time	VM left commented out; later moved behind an off-by-default switch	Capacity is a real operational concern; reserve capacity in production
5	Peering "inconsistent result" after a region change	Peering existed in Azure but was no longer tracked in state	terraform import to reattach it	terraform import reconciles real resources with state
6	git push rejected: 227 MB provider binary	.terraform/ was committed before a .gitignore existed	.gitignore, then git filter-branch to purge history, then force push	Create the .gitignore before the first commit
7	Subscription disabled; "free credits expired"	Free accounts are disabled at 30 days or when the $200 credit is spent	Upgraded to pay-as-you-go; terraform plan then showed "No changes"	Know your subscription type. State in a separate storage account survived, so recovery was a plan, not a rebuild
8	terraform plan prompted for subscription_id	The gitignored terraform.tfvars was missing after the repo was reorganized	Passed it through TF_VAR_subscription_id	Environment variables keep identifiers out of files and out of Git

#	Symptom	Root cause	Resolution	Lesson
9	Module not installed before any plan or state command	New modules must be installed by terraform init first	Ran terraform init, then re-ran the state backup and plan	Re-run init after adding modules, and verify a backup is non-empty
10	spokes.auto.tfvars ignored by Git	.gitignore excluded every *.tfvars file	Added !spokes.auto.tfvars below the *.tfvars line	A secret-hygiene rule can silently hide a needed file; confirm with git status

6. Cost and Governance

A hands-on cloud project is also a cost-control exercise. Practices applied here:

- **Billable resources are optional.** The VM and Bastion default to off, and the private endpoint is set per environment.
- **Plan as evidence.** A plan that proves the design costs nothing and leaves nothing running.
- **Timed demos.** Anything billed hourly is applied, captured, and destroyed in one sitting.
- **Budgets alert but do not stop spending.** Pay-as-you-go has no hard cap, so a low-threshold budget alert in Cost Management (budgets are free) belongs in place before any paid resource.
- **Know the billing model.** Bastion bills hourly from creation until deletion whether or not anyone connects, which is why it is a poor "leave it running" service.

7. Business and Career Alignment

Federal IT contractors increasingly expect cloud engineers to build and maintain infrastructure through code. The practices in this project line up with several frameworks (supporting, not certifying, compliance):

- **NIST 800-53 CM-2 and CM-3** (baseline configuration, change control): code in Git plus reviewed plans are a verifiable baseline and change record.
- **NIST 800-53 SC-7 and AC-17** (boundary protection, remote access): default-deny subnets, private endpoints, and a managed access path instead of open management ports.
- **FedRAMP and CMMC**: documented, reproducible, auditable configurations.
- **Zero Trust**: least-privilege network paths and no implicit trust between environments, at the infrastructure layer.

Skill	Evidence in this project
Infrastructure as Code	20 resources defined and managed through Terraform, organized into three reusable modules
Safe refactoring	14 resources moved to modules with moved blocks and a plan showing 0 destroyed
Azure networking	Hub-and-spoke topology, bidirectional peering, subnet design, NSG priority logic
Cloud security	Default-deny NSGs, private endpoint, public access and anonymous blob access disabled, TLS 1.2, no public VM IPs
Verification mindset	Found a gap between documentation and the portal, fixed it in code, and proved it with before and after evidence
Cost awareness	Off-by-default billable services, SKU trade-off analysis, subscription recovery
Troubleshooting and documentation	10 documented issues with root cause and lesson; README plus two PDF versions

Certification	Relevance
AZ-900 (passed Sept 24, 2026)	Resource groups, VNets, storage, subscriptions, and security concepts used hands-on
CompTIA Security+ (passed Aug 20, 2026)	Segmentation, least privilege, defense in depth, secure configuration
AZ-104 (next)	Virtual networking, NSGs, peering, private endpoints, and storage management make up much of the exam

8. What I Would Do Differently Next Time

Consolidated from the component sections, in the order I would act on them if I rebuilt this from scratch:

- Write the IP plan first, including the Bastion and firewall subnets, and keep everything in one region.
- Create the .gitignore (and the tfvars exception) before the first commit.
- Build the modules before the second environment, and build every environment from them.
- Use one state file per environment, with versioning and soft delete on the state account.
- Ship the private endpoint and its private DNS zone together.
- Make outbound NSG rules truly default-deny and turn on flow logs.
- Set a budget alert before creating anything billable, and keep paid services behind switches.
- Run fmt, validate, and plan automatically on every pull request.

Next steps

Action	Purpose
Private DNS zone, VNet link, and zone group	Complete the private endpoint so the storage name resolves to its private IP
Timed Bastion and VM demo	Prove hub-to-spoke private access across peering, capture screenshots, destroy
Per-environment state	Contain the blast radius of a bad apply
CI for fmt, validate, plan	Automatic checks on every change
Azure Firewall and routes (if inspection is required)	Turn spoke isolation into inspected spoke-to-spoke traffic
AZ-104	Certification path that covers this project's networking domain

9. Conclusion

This project shows that a secure Azure environment can be defined entirely in code and rebuilt on demand. Version 2 goes further: the code is now modular, a new environment is a data change, the documentation has been checked against what is actually deployed, and optional billable services are controlled by switches instead of left running.

The problems along the way (a disabled subscription, a storage account that did not match its documentation, SKU capacity, state moves, and Git history) are the everyday work of infrastructure engineering. Finding them, fixing them in code, and proving the fix is the job.

References: Microsoft Learn, Azure Bastion SKU comparison (learn.microsoft.com/azure/bastion/bastion-sku-comparison); Azure Bastion configuration settings (learn.microsoft.com/azure/bastion/configuration-settings); Azure free account FAQ (azure.microsoft.com/free/free-account-faq).

Documentation prepared by Bryan Sykes, October 26, 2026.